

15. Strong Consistency & Scaling Out: Konsensus

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

15.1 Vorschau

- Grundlagen
- FLP-Theorem
- Paxos
- Chubby Lock Service
- Die byzantinischen Generäle



Griechische Insel Paxos

15.2 Grundlagen



Begriffe

- Einigung (engl. consensus, agreement)
 - N Server müssen sich auf „etwas“ einigen
 - Wird oft auf 0 oder 1 reduziert
- Konsistenz (engl. consistency)
 - Konsistenz definiert die Reihenfolge wie Ereignisse für alle beteiligten Server sichtbar werden
 - Dies ist im Prinzip auch eine Einigung bezüglich der Reihenfolge von Ereignissen
 - Beide Begriffe meinen (fast) das Gleiche

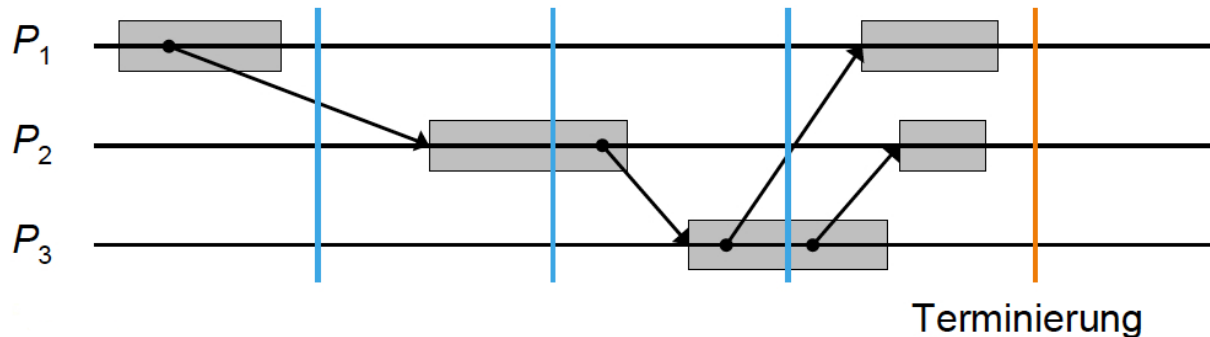


Annahmen an das System

- Es gibt N beteiligte Server (global bekannt)
- Darunter gibt es maximal M fehlerhafte Server
- Jeder Server kann mit jedem kommunizieren
 - Kommunikation ist zuverlässig $\rightarrow$ TCP
 - Absender von Nachrichten ist immer sicher feststellbar $\rightarrow$ digitale Signatur

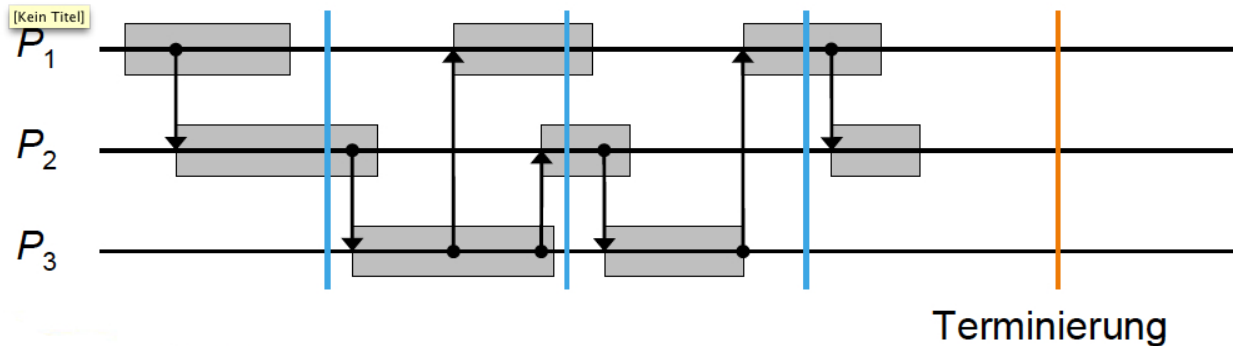
Asynchrones Kommunikationsmodell

- Prozesse sind aktiv oder passiv
 - Nur aktive Prozesse versenden Nachrichten
 - Ein aktiver Prozess kann jederzeit passiv werden
 - Ein passiver Prozess kann nur durch eine Nachricht reaktiviert werden



Synchrones Kommunikationsmodell

- Unterschied zum asynchronen Modell
 - Nachrichten haben keine Laufzeit
 - Dies wird durch synchrone Kommunikation nachgebildet
→ Sender wird blockiert, bis Nachricht empfangen wurde



FLP-Theorem

- Fischer, Lynch und Paterson haben grundsätzlich gezeigt, dass es für das Einigungsproblem
 - im asynchronen System-Modell, wenn Rechnerausfälle auftreten können, keine total korrekte Lösung gibt.
 - Der Beweis zeigt, dass es keinen asynchronen Einigungsalgorithmus (für einen Ein-Bit-Wert) gibt, der garantiert terminiert, wenn Prozessabstürze auftreten.
 - Das Theorem bezieht sich nur auf gutartige Fehler, also nicht auf byzantinische Fehler
- Literatur:
 - „Impossibility of distributed consensus with one faulty process“, M. J. Fischer, et.al., Journal of the ACM Volume 32, Issue 2, April 1985.



FLP-Theorem – Bedeutung für die Praxis

- Fehlertolerante Einigungsalgorithmen sind möglich und deren Korrektheit kann bewiesen werden, aber man kann nicht beweisen, dass sie auch terminieren
- Die Situation, dass der Algorithmus nicht terminiert ist unwahrscheinlich, kann aber eintreten.
- Beispiel: ein partiell korrekter Algorithmus ist „Paxos“
 - Terminierung kann hier aber nicht garantiert werden, ist aber selten
 - Siehe später



15.3 Konsensus in synchronen Systemen



Einigung mit perfekter Ausfallerkennung

- Perfekter Ausfallerkennung
 - Im synchronen Modell einfach, z.B. mit Heartbeat-Nachrichten und Timeouts
- Einigung: erste Idee mithilfe eines Koordinators
 - Koordinator sendet seinen Wert an alle
 - Alle anderen Knoten übernehmen diesen Wert
- Problem: Ausfall des Koordinators
 - Es ist zu verhindern, dass ein neuer Koordinator einen inkonsistenten Wert verbreitet, falls alter Koordinator einen Wert bereits verbreitet hat



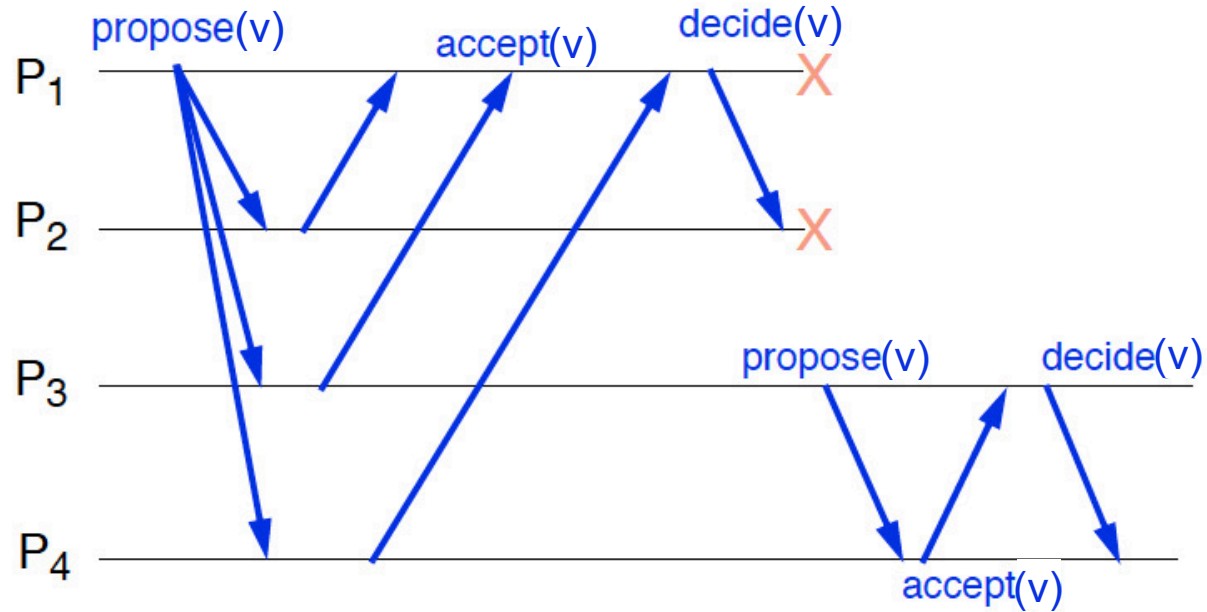
Zwei Phasen-Commit-Protokoll (2PC)

- Phase 1: Koordinator stellt sicher, dass „sein“ Wert einzig möglicher Entscheidungswert ist:
 - Wert v als Vorschlag an alle senden → **propose (v)**
 - Jeder übernimmt diesen Wert v als eigenen Vorschlagswert (für den Fall, dass er selbst Koordinator werden sollte)
 - Der Koordinator wartet auf eine Bestätigung **accept (v)** von allen nicht ausgefallenen Knoten (durch synchrones Modell ist max. Zeit bekannt)
- Phase 2: Koordinator verbreitet die Entscheidung für seinen Wert **decide (v)**
 - Falls Koordinator beim Verbreiten der Entscheidung ausfällt, ist sichergestellt, dass der nächste Koordinator den gleichen Wert propagiert, also keine Inkonsistenz entstehen kann



2PC: Beispiel

- Beispielablauf



Hans P. Reiser, Universität Ulm

2PC: Bemerkungen

- Funktioniert nur bei perfekter Ausfallerkennung & gutartigem Fehlermodell
 - In der Praxis nicht machbar
- Probleme bei einer nicht perfekter Ausfallerkennung:
 - Koordinator wartet auf Antwort (accept) von allen nicht ausgefallenen Knoten.
 - Bei einem nicht erkannten Ausfall wartet der Koordinator ewig
 - Bei einem fälschlicherweise als ausgefallen erkannten Knoten können Inkonsistenzen entstehen
 - Aktivierung eines neuen Koordinators hängt von der exakten Erkennung des Ausfalls des alten Koordinators ab
 - Bei fehlerhafter Ausfallerkennung kann es mehrere Koordinatoren zugleich, oder keinen Koordinator geben



15.4 Konsensus in asynchronen Systemen



Allgemeines

- Ziel: Verbesserung des zuvor beschriebenen 2 PC-Protokolls, sodass Konsensus auch bei einer nicht-perfekten Ausfallerkennung funktioniert
- Statt alle Knoten zu berücksichtigen wird mit einem Quorum gearbeitet
 - Neuer Koordinator „weiß“ alte Werte nicht zwingend, aber kann sie durch Befragung eines Quorums ermitteln
 - Statt aller Knoten muss nur Quorum verfügbar sein → vermeidet Blockierung
- Bestimmung des Koordinators
 - Inkonsistenzen werden vermieden, selbst wenn es zeitweise mehrere Koordinatoren gleichzeitig geben sollte



Allgemeines

- Um maximal t simultane Serverausfälle maskieren zu können, werden $2 \cdot t + 1$ Server insgesamt benötigt
- Crash-Recovery-Fehlermodell
 - **Crash:** Korrekte Knoten dürfen zwischendurch ausfallen
 - Schicken aber keine fehlerhafte Nachrichten (fail stop)
 - **Recovery:** Relevanter Zustand wird in stabilem Speicher gesichert und wird nach Neustart wieder eingelesen
 - kein Zustandsverlust durch Serverausfall
 - Aber auch hier keine byzantinische Fehler erlaubt



Grundidee (1)

- Koordinator versucht durch Wiederholung, eine Einigung für einen Wert durchzusetzen
 - Wiederholungen sind Basis für Crash-Recovery-Modell
- In ungünstigen Situationen kann es mehrere Koordinatoren gleichzeitig geben, aber keine Inkonsistenzen, siehe später
- Es können mehrere Einigungsversuche Erfolg haben; in diesem Fall verwenden aber alle den gleichen Vorschlagswert



Grundidee (2)

- Manchmal sind nicht alle Replikate erreichbar
 - Schreib-Befehle würden u.U. ewig auf Antwort warten
 - Lese-Befehle würden u.U. den falschen Wert zurückgeben
- Lösung: Quoren
 - $Q_W + Q_R > N$: Writes und Reads überlappen
 - $Q_{W1} + Q_{W2} > N$: Zwei Writes überlappen (wichtig, um Konflikte zu erkennen)

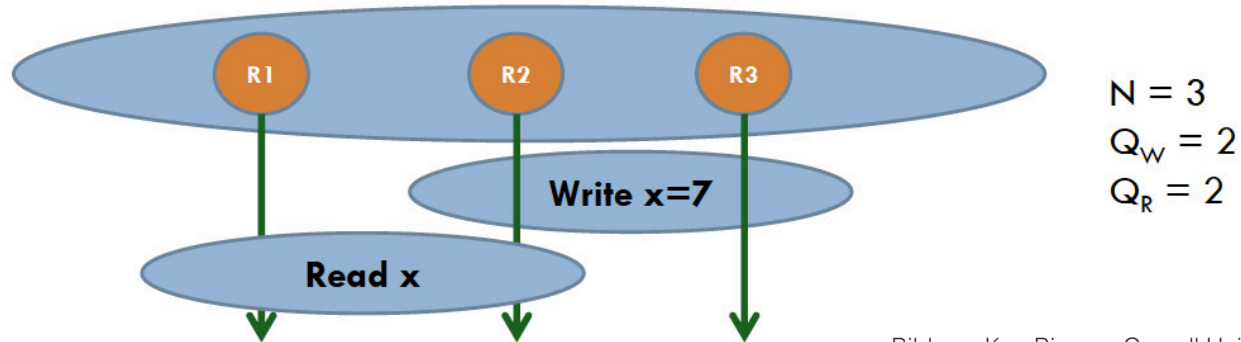


Bild von Ken Birman, Cornell University

Grundidee (3)

- Schreib-Befehle werden per 2-Phasen-Commit (2PC) durchgeführt (propose & accept)
 - Wenn Koordinator abstürzt haben Replikate einen anhängigen Schreib-Befehl, kennen aber nicht den Ausgang
 - Deswegen muss ein 2PC notfalls wiederholt
- Bem.: Kein 2PC wie bei verteilten Transaktionen, da hier nur auf einem Quorum gearbeitet wird und nicht auf allen Replikaten



Terminologie (1)

- Paxos unterscheidet verschiedene Rollen:
 - Dennoch kann ein Prozess mehrere Rollen gleichzeitig ausführen
 - Rollen vereinfachen die Logik und das Verständnis
- Rollen:
 - **Proposer**: Anwendung die zu Paxos „spricht“
 - **Coordinator**: Anführer, der das Protokoll steuert
 - **Acceptor**: ein Teilnehmer der Abstimmung/Protokolls (Replikat)
 - **Learner**: Paxos „spricht“ zur Anwendung (Ergebnis)



Terminologie (2)

- Der Proposer schickt eine Anfrage an das Paxos-System (Paxos erhält Nachricht, vergleiche Post)
- Paxos „denkt nach“ und entscheidet bzgl. der Nachricht (repliziert Daten und legt die Auslieferungsordnung fest)
- Wenn dies erledigt ist, können Learner den Befehl ausführen

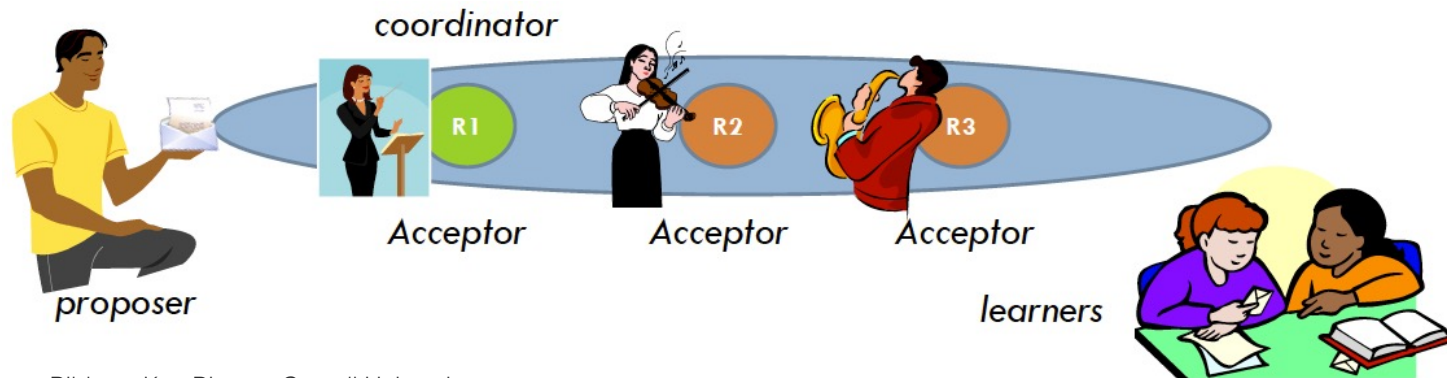


Bild von Ken Birman, Cornell University

Rolle des Koordinators

- Phase-1: „prepare“
 - schickt „prepare“-Nachricht an die Acceptors, um eine neue Wahl vorzubereiten
 - Wird u.U. mehrmals wiederholt
- Phase-2: „accept“
 - Hier wird entschieden, welcher Wert angenommen wird
 - Koordinator verschickt hierzu „accept“-Nachricht
 - U.U. bricht der Koordinator ab und wiederholt ab Phase-1
- Phase-3: „learn“
 - Erfolgreich entschiedene Werte, schickt der Koordinator an die Learners



Rolle der Acceptors

- Verwalten empfangene Werte und nehmen an Wahlen teil

Wahlnummern

- Engl. ballot numbers → müssen eindeutig und aufsteigend sein
- Ziel ist Einigung, dass ein bestimmter Wert angenommen wird
 - Wenn zwei Koordinatoren konkurrieren,
so gewinnt derjenige mit der größeren Wahlnummer
 - Der Verlierer wiederholt mit größerer Wahlnummer (ggf. nach einer Wartezeit)
- Kann mehrere Runden benötigen (theoretisch beliebig viele)
- Diese Runden werden anhand der Wahlnummer unterschieden



Vollständige Phase 1

- Proposer schickt **propose (v)** an den Koordinator
- Koordinator schickt **prepare (b)** an alle Acceptors (b: Ballot-Nummer)
- Beim Empfang prüft ein Acceptor:
 - Wenn er zuvor keine `prepare (b')`-Anfrage gesehen hat, mit $b' > b$, so antwortet er mit **promise (b, v*)**
 - v^* : bereits mit `accept ()` beantworteter Wert, aus einer früheren Wahlrunde b^* , mit $b^* < b$ oder
 - $v^*=0$, falls bisher keine Wahl stattfand.
 - Andernfalls antwortet er nicht
- Der Koordinator:
 - Bricht ab, falls er kein Quorum mit `promises` erreicht und wiederholt mit einer größeren Ballot-Nr. $b'' > b'$
 - Ansonsten geht er in Phase 2

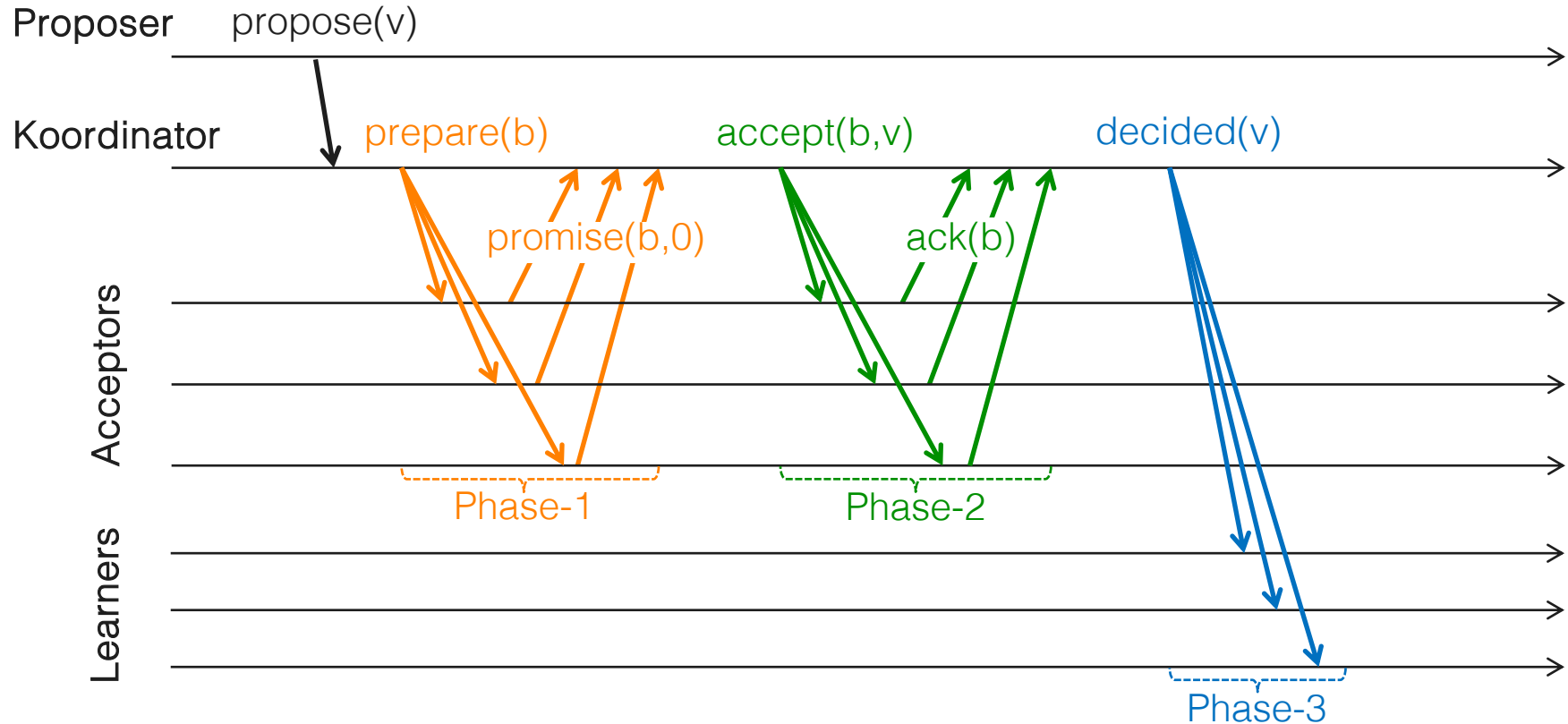


Vollständige Phase 2

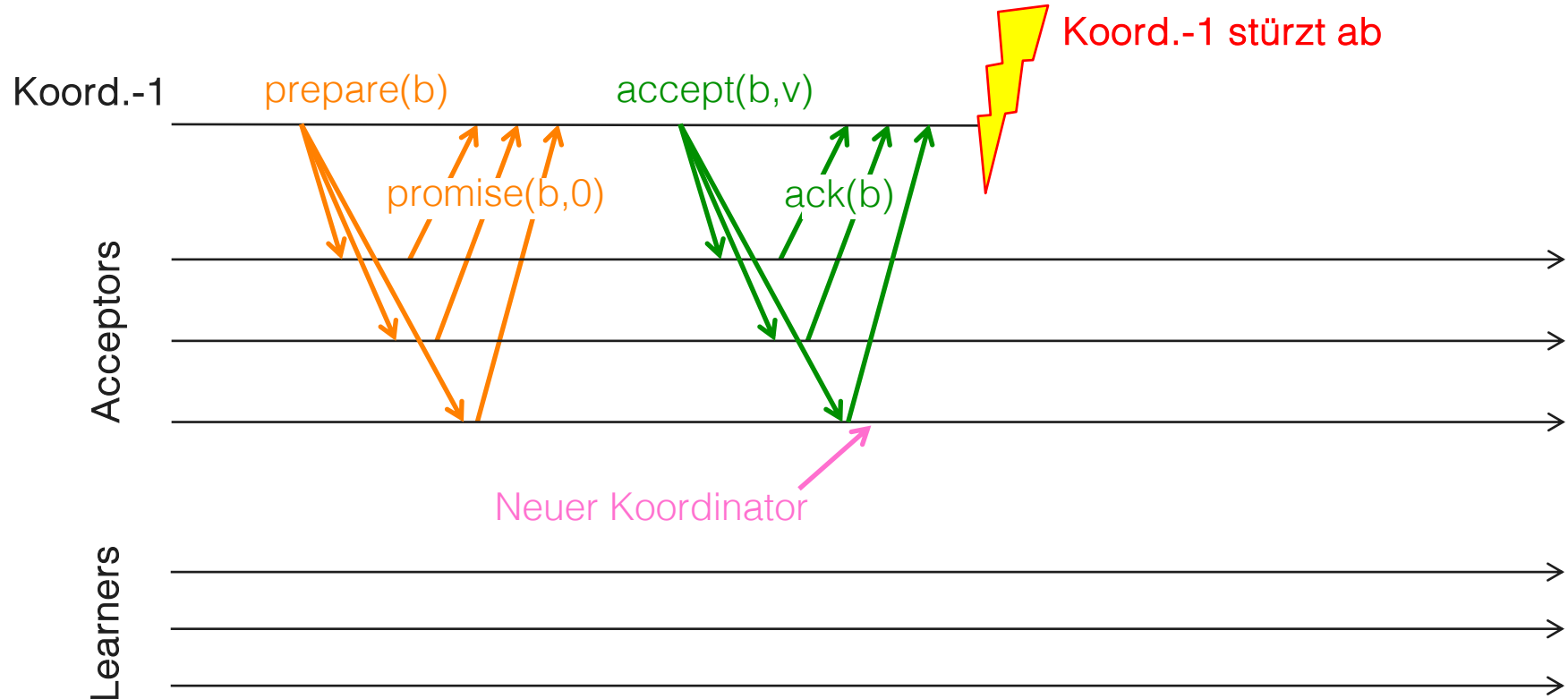
- Koordinator fordert Acceptors auf, den Wert v^* anzunehmen, wozu er die Nachricht **accept** (b , v^*) schickt.
 - v^* ist der Wert aus den `promise()`-Antworten aus Phase-1
 - Falls alle mit $v^* = 0$ geantwortet haben, so wird v verwendet
- Acceptors:
 - Antworten mit `ack(b)`, falls sie keine neue Wahlnummer $b' > b$ empfangen haben, ansonsten tun sie nichts.
- Koordinator:
 - Wiederholt die Wahl ab Phase 1, falls er kein Quorum erhält
 - Ansonsten geht er in Phase-3



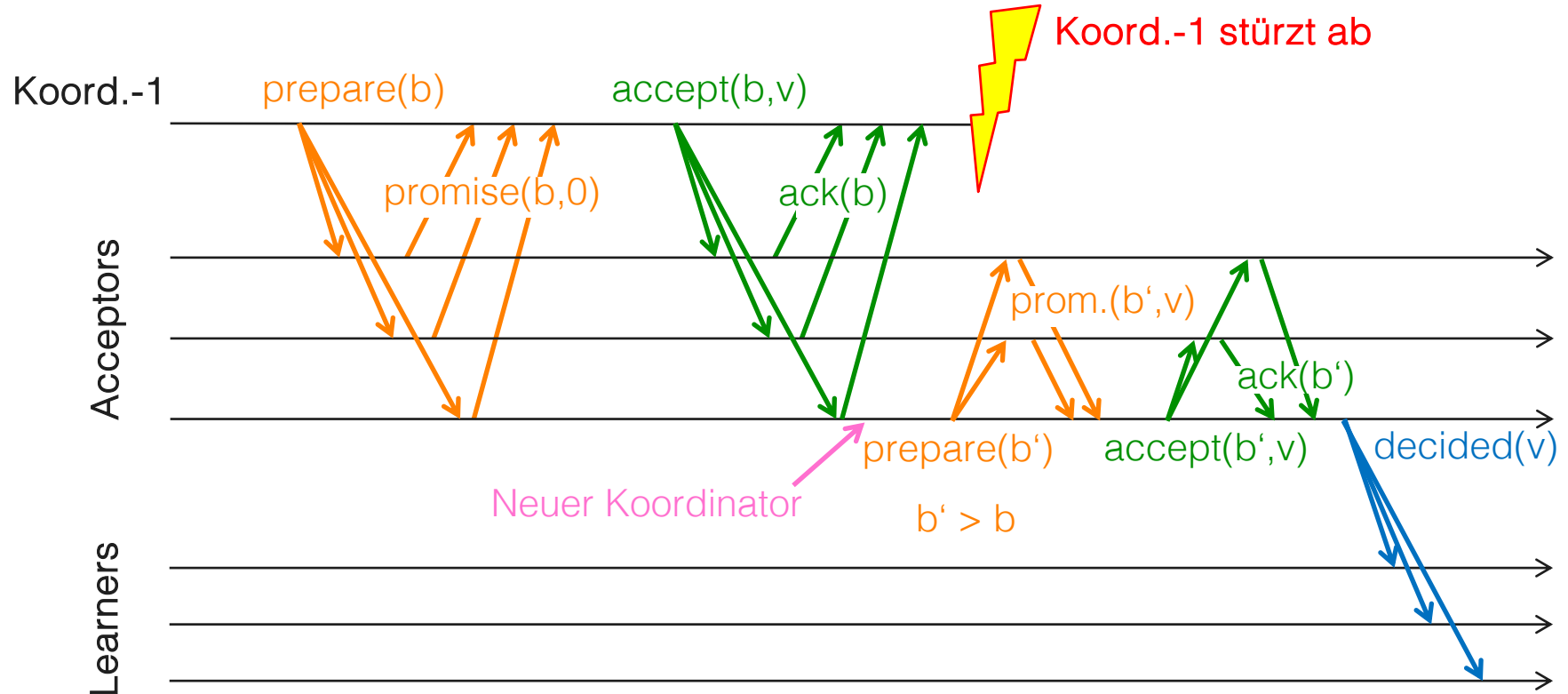
Fehlerfreier Ablauf (keine Wahl zuvor)



Fehler: Koordinator stürzt ab (keine Wahl zuvor)



Fehler: Koordinator stürzt ab (keine Wahl zuvor)

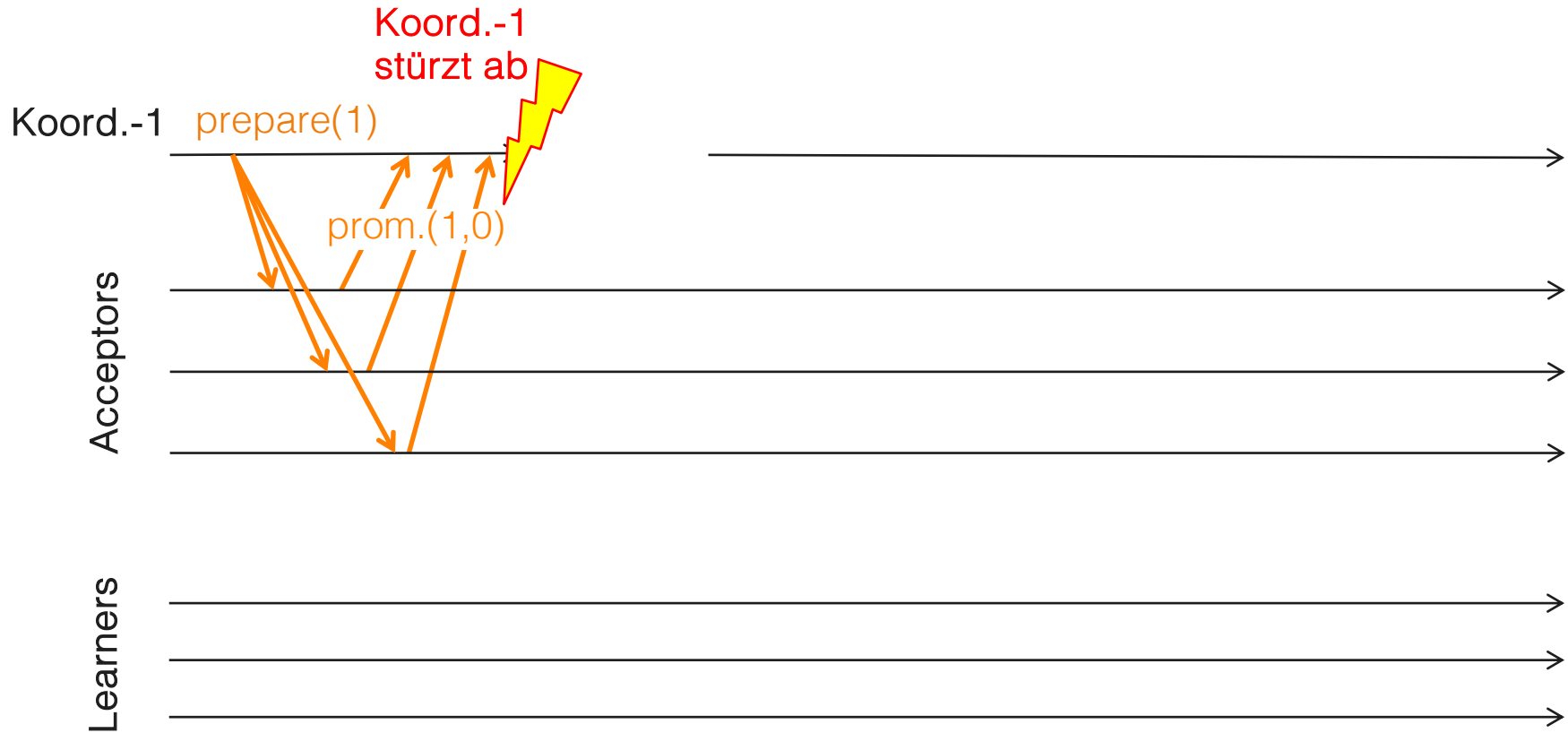


Weitere Fehler

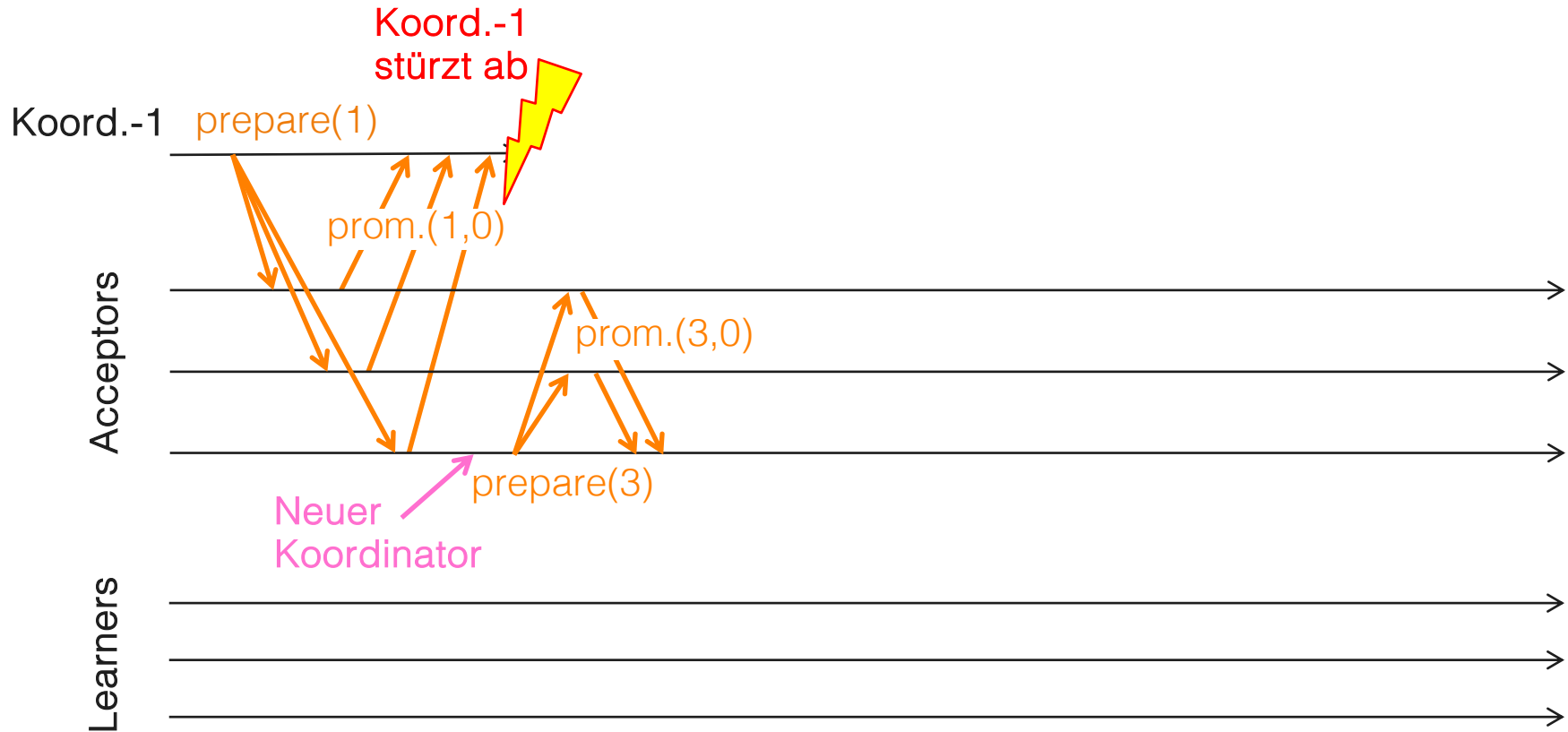
- Acceptor stürzt ab:
 - Solange Quorum erreichbar ist, kein Problem
 - Ansonsten Timeout und Koordinator startet wieder mit Phase 1
- Learner:
 - Falls alle Learner abstürzen, muss der Client eine Dummy-Wahl ausführen, um den entschiedenen Wert zu ermitteln
- Netzwerkpartitionierung
 - Nur in einer Partition ist ein Quorum erzielbar
 - Andere Partition ist blockiert
- Alle beteiligten Prozesse können jederzeit abstürzen
 - Wahl verläuft immer eindeutig, allenfalls verzögert



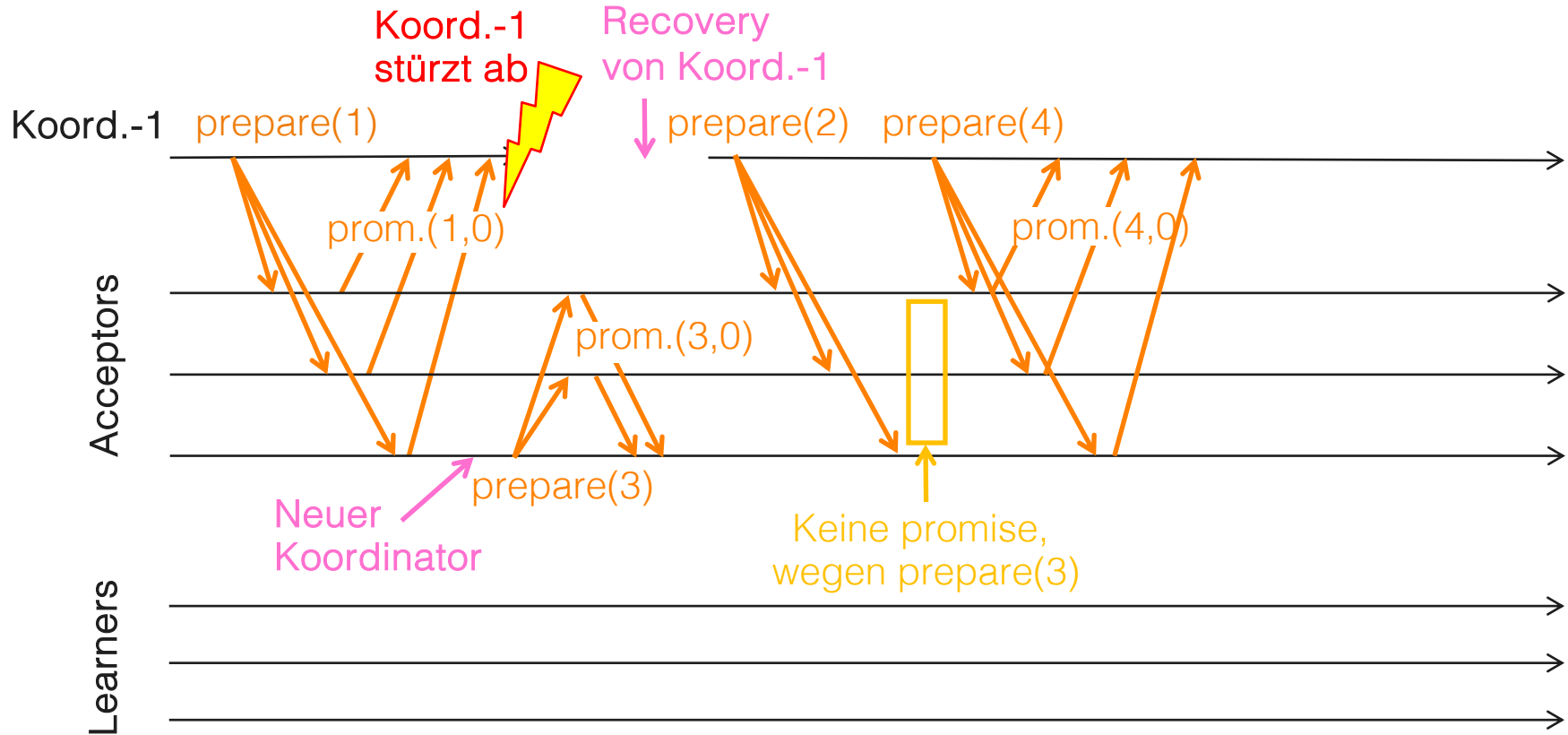
Fehler: Zwei Koordinatoren „duellieren“ sich



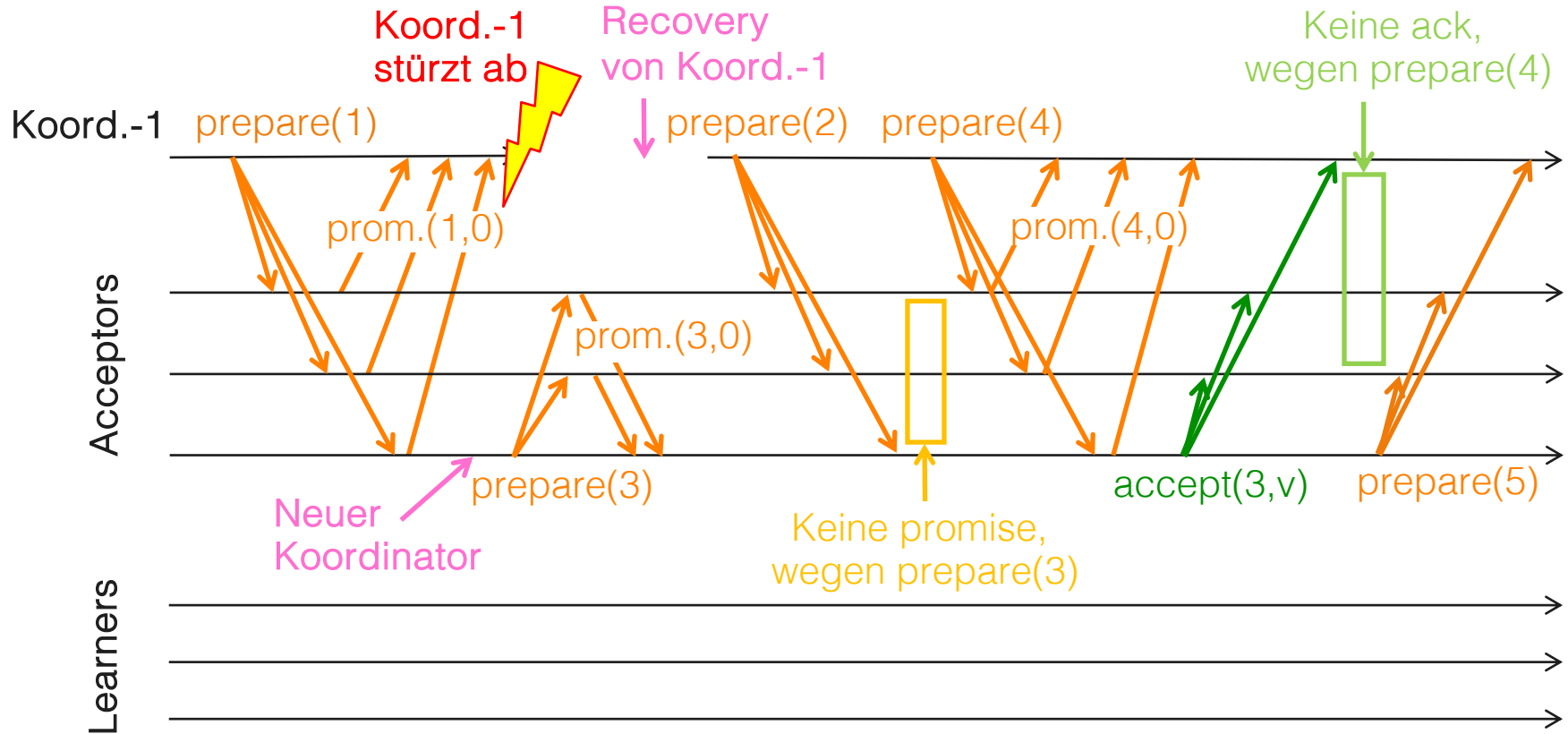
Fehler: Zwei Koordinatoren „duellieren“ sich



Fehler: Zwei Koordinatoren „duellieren“ sich



Fehler: Zwei Koordinatoren „duellieren“ sich



Ermittlung des aktuellen Koordinators

- Es ist wünschenswert, dass
 - alle Server stets den gleichen Server als Koordinator sehen
 - bei Ausfall des Koordinators schnell ein neuer bestimmt wird
- Für die Konsistenz ist es nicht notwendig, dass beide Bedingungen immer eingehalten werden (unmöglich in asynchronen Systemen)
- Die Original-Beschreibung von Paxos legt hier keinen bestimmten Algorithmus fest → z.B. Bully-Algorithmus (Server mit größter ID)

Ausfall und Wiederanlauf eines Acceptors

- Wichtige Informationen müssen nach einem Ausfall und Wiederanlauf eines Server wiederhergestellt werden → Speicherung auf Disk
 - Die größte Wahlnummer b , die ein Acceptor in einer `prepare`-Nachricht gesehen hat
 - Die größte Wahlnummer b , bei der eine `accept`-Nachricht gesehen wurde, und der akzeptierte Wert v

Bemerkungen

- Es gibt auch eine Erweiterung, die bei den Acceptors **nacks** verwendet
- MultiPaxos:
 - Oft ist eine Abfolge von Entscheidungen („Folgen“) notwendig, d.h. mehrfache Ausführung von „Basic“ Paxos-Instanzen
 - Optimierung:
 - Phase 1 nur ein Mal durchführen
 - Phase 2 N Mal
 - Im Fehlerfall muss wie bisher auch ab Phase 1 wiederholt werden



- „The Part-Time Parliament“, L. Lamport, Technical Report, DEC Systems Research Center, Sept. 1989; auch in: ACM Transactions on Computer Systems, Volume 16, Issue 2, 1998
- „Paxos Made Simple“, L. Lamport“, ACM SIGACT News (Distributed Computing Column) 32, 4, Dec. 2001
- „Paxos for System Builders: An Overview“, Jonathan Kirsch, Yair Amir, Workshop on Large-Scale Distributed Systems and Middleware, 2008.

15.5 Chubby Lock Service (Google)



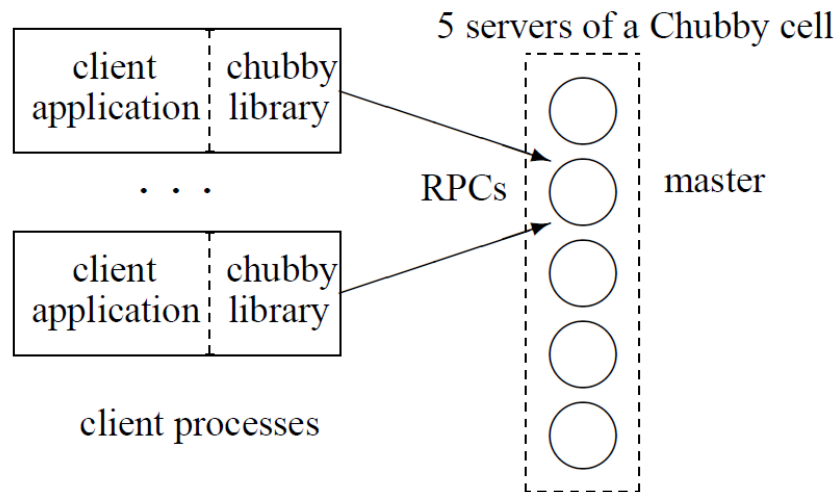
Allgemeines

- Zuverlässiger verteilter Sperrdienst & Speicher für kleine Dateien = chubby files (Konfigurationsdaten von Diensten)
 - Freiwillige Sperren (engl. advisory lock) auf Dateien
 - Ereignismitteilung (z.B. Dateiinhalt geändert)
 - Sicherheit (UNIX-typische Rechte)
- Chubby implementiert intern Paxos
- Verwendet in verschiedenen Google-Diensten, z.B. BigTable, GoogleFS, ...



System-Struktur

- **Chubby-Cell**: besteht aus meist 5 Servern (jeweils in verschiedenen Racks)
- **Master**: bestimmt durch Server einer Chubby cell mithilfe von Paxos
 - periodisch, immer für einige Sekunden
 - Details siehe Literatur
- **Clients**: greifen über den Master zu
 - Dieser verteilt Updates der Clients per Paxos an die Replikate innerhalb der Chubby-Cell
 - Lesen erfolgt vom Master (ohne Paxos-Abstimmung)



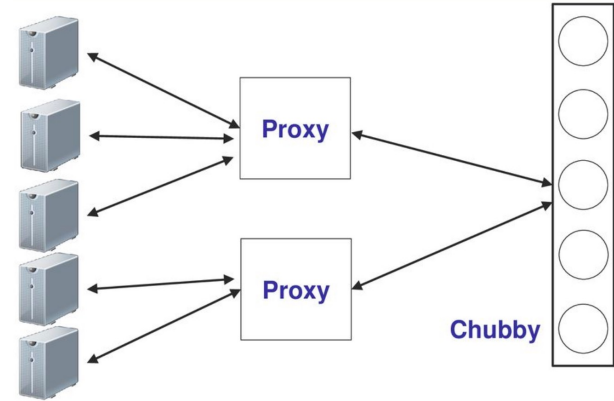
Caching

- Ziel: Lesezugriffe über das Netzwerk reduzieren
- Klienten cachen Chubby-Dateien (Daten und Metadaten) lokal
 - Write-through Cache
- Invalidierung durch den Master
 - Speichert Liste der Klienten und deren gecachte Dateien
 - Master führt Änderungen in einer Datei nur durch, nachdem Invalidierungen von allen Klienten bestätigt wurden, die die betreffende Datei geöffnet haben oder Client-Leases von Klienten abgelaufen sind → strenge Konsistenz



Optimierungen zur Verbesserung der Skalierbarkeit

- Mehrere Proxies für eine Cell:
 - Bündeln Anfragen von mehreren Klienten
 - Entlasten somit Master
 - Eine Cell bedient bei Google so bis zu 10.000 Clients
- Partitionierung über mehrere Cells
 - Namensraum wird partitioniert zwischen den Cells



- Chubby-Lock Service:
 - „The Chubby lock service for loosely-coupled distributed systems“, M. Burrows, Symposium on Operating Systems Design and Implementation, USA, 2006.
- Alternative Open-Source Implementierung: Apache Zookeeper
 - „ZooKeeper: wait-free coordination for internet-scale systems“, P. Hunt et. al., USENIX ATC, 2010.
- Neuere Open-Source Alternative: Apache Ratis
 - „In search of an understandable consensus algorithm“, O. Ongaro et. al., USENIX ATC, 2014.



15.6 Byzantinischer Konsensus



Die byzantinischen Generäle

- L. Lamport, R. Shostak, M. Pease, „The Byzantine Generals Problem“, ACM Transactions on Programming Languages and Systems, Volume 4 Issue 3, July 1982
- Publikation zeigt, dass Konsensus möglich ist
 - Wenn bei mind. $3m + 1$ Prozessen maximal m fehlerhaft sind
 - $2/3$ aller Prozesse müssen korrekt arbeiten und die Gesamtanzahl der Prozesse muss >3 sein
 - Maskiert byzantinische Fehler (also auch böses Verhalten)
 - Hier betrachtet für ein synchrones Systemmodell



Die byzantinischen Generäle

- n Generäle ($n > 3$), m davon sind Verräter
- Anführer gibt Befehl $b \in \{0, 1\}$ (z.B. Angriff=1)
- Die anderen Generäle sollen für b den gleichen Wert verwenden
- Frage: Einigung auf angreifen (1) oder abwarten (0)?
 - Der Anführer ändert seine Entscheidung nicht.
- Bem.: Falls der Anführer der Verräter ist, so schickt er unterschiedliche Befehle an die anderen.



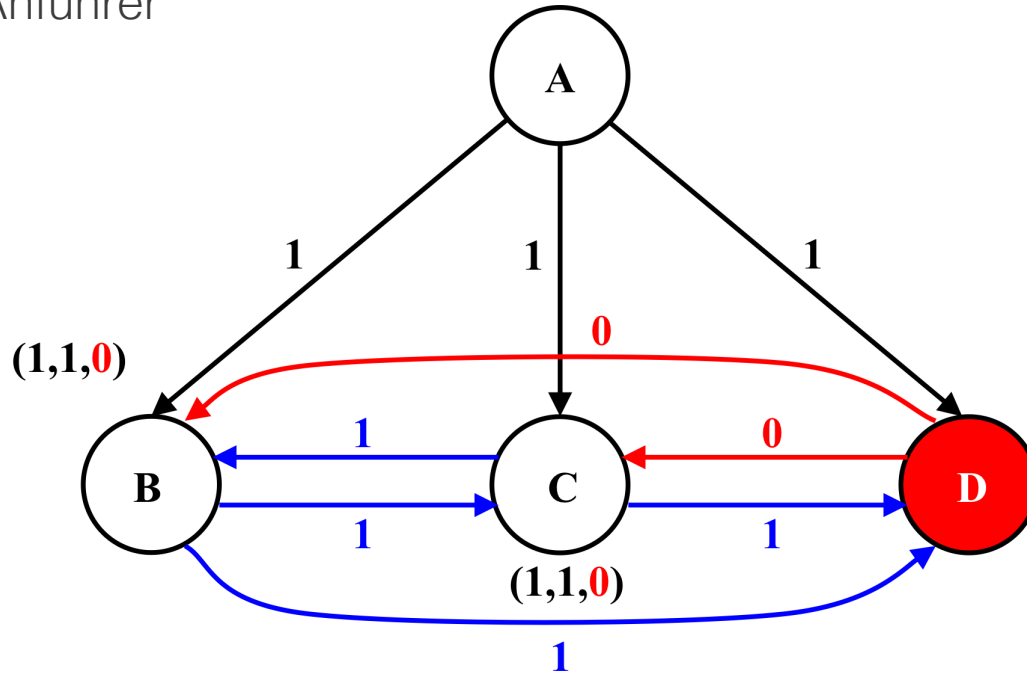
Die byzantinischen Generäle für $m=1$

1. Anführer A sendet Befehl an alle Generäle
2. Jeder General teilt allen anderen mit,
welchen Befehl er von dem Anführer erhalten hat
3. Jeder General trifft aus den erhaltenen Werten eine Mehrheitsentscheidung.



Die byzantinischen Generäle für $m=1$

- Verräter (Knoten D) versucht Einigung zu verhindern
- A ist der Anführer



Die byzantinischen Generäle für $m > 1$

- Kann auf größere m erweitert werden, indem Algorithmus iterativ ausgeführt wird $\rightarrow m+1$ Runden (synchrones Modell)
 - Voraussetzung: Absender einer Nachricht ist identifizierbar
- Ablauf:
 - Anführer schickt Befehl als Nachricht an alle n Generäle
 - Empfang einer Nachricht:
 - Jeder General verhält sich wie der Anführer und schickt empfangene Nachricht/Befehl zu allen anderen Generälen (außer an den Anführer von dem der Befehl kam)



Die byzantinischen Generäle für $m > 1$

- 1. Runde: 1 Anführer verschickt $n-1$ Nachrichten
- 2. Runde: $n-1$ Anführer verschickt je $n-2$ Nachrichten
- 3. Runde: $(n-1)(n-2)$ Anführer verschickt je $n-3$ Nachrichten
- ...
- $O(n^{m+1})$ Nachrichten



Exponentielle Zunahme der Anzahl der Nachrichten

- $n = 4, m = 1 \rightarrow 2$ Runden
 - $3 + 3 \cdot 2 = 3 + 6 = 9$ Nachrichten
- $n = 7, m = 2 \rightarrow 3$ Runden
 - $6 + 6 \cdot 5 + 6 \cdot 5 \cdot 4 = 156$ Nachrichten
- $n = 10, m = 3 \rightarrow 4$ Runden
 - $9 + 9 \cdot 8 + 9 \cdot 8 \cdot 7 + 9 \cdot 8 \cdot 7 \cdot 6 = 3.609$ Nachrichten
- $n = 13, m = 4 \rightarrow 5$ Runden
 - $13 + 13 \cdot 12 + 13 \cdot 12 \cdot 11 + 13 \cdot 12 \cdot 11 \cdot 10 + 13 \cdot 12 \cdot 11 \cdot 10 \cdot 9 = 108.384$ Nachrichten

n = Anzahl Generäle
 m = Anzahl der Verräter



Nachrichtenbaum $n=7, m=2$

- Jeder General hat solch einen Baum; hier für Knoten 1
 - Anführer ist Knoten 0 → sendet „a“ ($m=2 \rightarrow 3$ Runden)
 - Mehrheitsbildung von unten nach oben

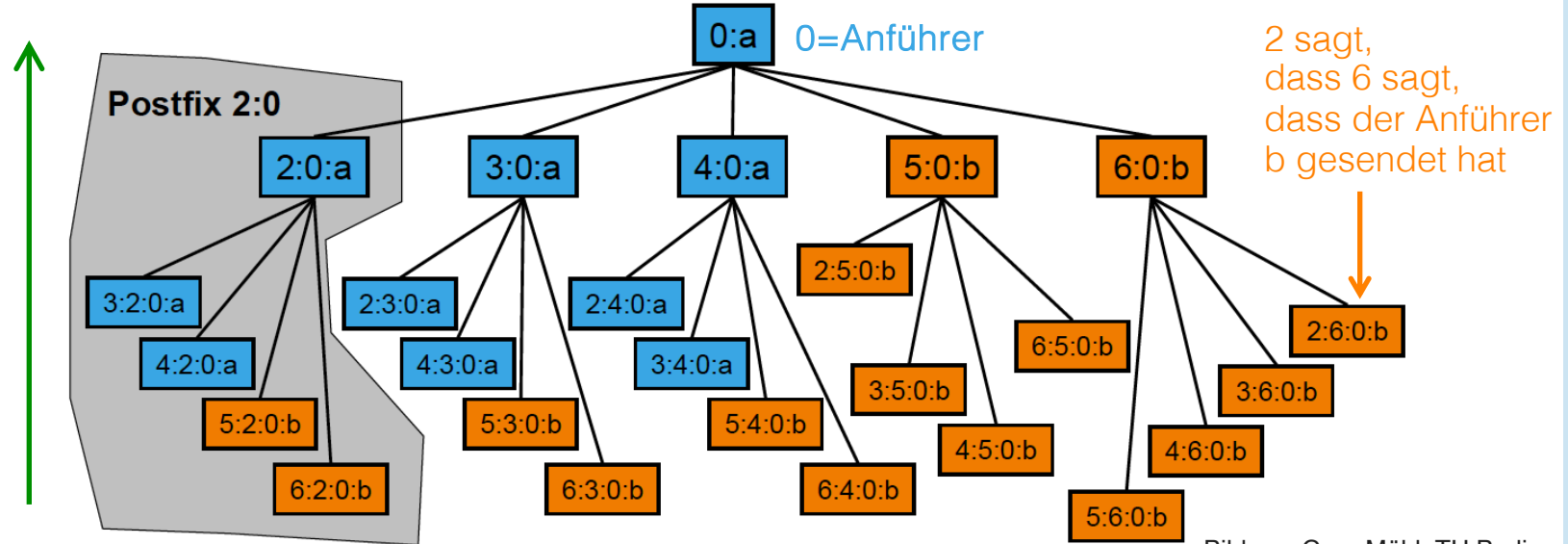


Bild von Gero Mühl, TU Berlin

Bemerkungen

- Treten gleich viele a wie b auf, so muss vorab definiert sein, welcher Wert dominiert, um wieder eine Einigung zu erzielen.
- FLP gilt auch hier!
 - Algorithmus terminiert u.U. nicht in einem asynchronen System
 - Vorgestellter Algorithmus war für ein synchrones System



15.6 Zusammenfassung



Zusammenfassung

- FLP-Theorem besagt, dass in einem asynchronen System keinen total korrekten Einigungsalgorithmus gibt
- Für die Praxis ist Paxos ein wichtiger Einigungsalgorithmus, welcher häufig verwendet wird, jedoch nur für ein gutartiges Fehlermodell gedacht ist
- Beispiele für Paxos-Implementierungen:
 - Chubby lock service (Google)
 - Zookeeper (Yahoo!), Open-Source
 - Autopilot (Microsoft) ...
 - Ein ähnlicher Algorithmus ist Raft: Apache Ratis, Open-Source
- Byzantinische Fehlertoleranz skaliert schlecht

